2주차 공통실습

업무 영역	Devops 자동화
진행 현황	작업완료
시작일	@2025년 8월 11일
마감일	@2025년 8월 12일
진행률	1
설명	-[O] Flask 앱(app.py) 작성 및 실행 (python app.py) -[O] 브라우저로 퍼블릭 IP:5000 접속 테스트 -[O] UFW 방화벽 설치 (sudo apt install ufw) -[O] UFW에서 80/22 포트 허용, 기타 포트 차단 실습 (sudo ufw allow 80, sudo ufw deny 8080) -[O] 보안 그룹 규칙 변경 실습 (포트 추가/삭제 후 접속 테스트) -[O] GitHub → EC2 수동 배포 실습 (선택) -[O] 배포 성공 화면 캡처 및 배포 흐름 간단 정리 -[O] CloudWatch 메트릭(기본) 확인 – CPU/네트워크 그래프
주차	2주차

1. Flask 앱(app.py) 작성 및 실행 (python app.py)

목적: 단순한 웹 서버 기능을 가진 Flask 앱을 작성하여 컨테이너 테스트용으로 사용

작업 내용

- EC2 접속(PUTTY)
- 프로젝트 폴더 생성 및 가성환경 설치

```
mkdir ~/test
cd ~/test

# Python venv 설치
sudo apt install python3-venv -y
```

```
# 가상환경 생성
python3 -m venv venv

# 가상환경 활성화
source venv/bin/activate
```

- flask 애플리케이션 작성

```
from flask import Flask, jsonify
import socket
import datetime


app = Flask(__name__)

# 기본 페이지
@app.route('/')
def hello():
    return "Hello from Flask on EC2!", 200

# ALB 헬스체크용
@app.route('/health')
def health():
    return jsonify(status="healthy"), 200

# 상태 확인 (운영자용)
@app.route("/status")
def status():
    return jsonify(
        app="autoscaling-demo",
        version="1.0.0",
        hostname=socket.gethostname(),
        time=datetime.datetime.utcnow().isoformat()
    ), 200
```

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

/: <퍼블릭 IP>:5000에 접속하면 "Hello from Flask on EC2!" 출력

/health: <퍼블릭 IP>:5000/health에 접속하면 "OK" 출력

- flask 설치 및 실행

```
pip install flask

python3 app.py
```

2. 브라우저로 퍼블릭 IP:5000 접속 테스트

목적: 작성한 FLASK 앱의 실행 상태 확인

- http://퍼블릭IP:5000/ 접속 테스트 확인



3. UFW 방화벽 설치 및 규칙 설정

목적: AWS 보안 그룹과 서버 방화벽 차이를 이해하고, 나중에 온프레미스 환경에서도 방화벽을 다룰 수 있도록 훈련.

- UFW 설치 및 상태 확인

```
sudo apt update
sudo apt install ufw -y

sudo ufw status # 기본 상태는 inactive (비활성)
```

- 기본 규칙 설정 (80/22 허용, 기타 포트 차단)

```
# SSH(22번 포트) 허용 – 안 하면 서버 접속이 끊길 수 있음
sudo ufw allow 22

# HTTP(80번 포트) 허용
sudo ufw allow 80

# 예시: 특정 포트(8080) 차단
sudo ufw deny 8080
```

- UFW 활성화 후 상태 확인

```
sudo ufw enable # y입력

sudo ufw status

To                         Action      From
--                         ------      ----
22                         ALLOW       Anywhere
80                         ALLOW       Anywhere
8080                       DENY        Anywhere
22 (v6)                    ALLOW       Anywhere (v6)
80 (v6)                    ALLOW       Anywhere (v6)
8080 (v6)                  DENY        Anywhere (v6)
```

```
#(v6)는 IPv6
```

4. AWS 보안 그룹 규칙 변경 실습

목적: AWS 보안 그룹이 서버 외부에서 트래픽을 차단/허용하는 역할을 이해

1. AWS 콘솔 접속
2. AWS 로그인 - EC2 - 보안 그룹 - 사용 중인 보안 그룹 ID 클릭
3. 인바운드 규칙 편집 클릭
4. 인바운드 규칙 추가로 사용자 지정 IP - 8080 - 내 IP 선택
5. 저장 클릭

5. GitHub → EC2 수동 배포 실습

목적: 깃허브의 내용을 clone해서 가져온다.

- ec2 접속 후 git 설치

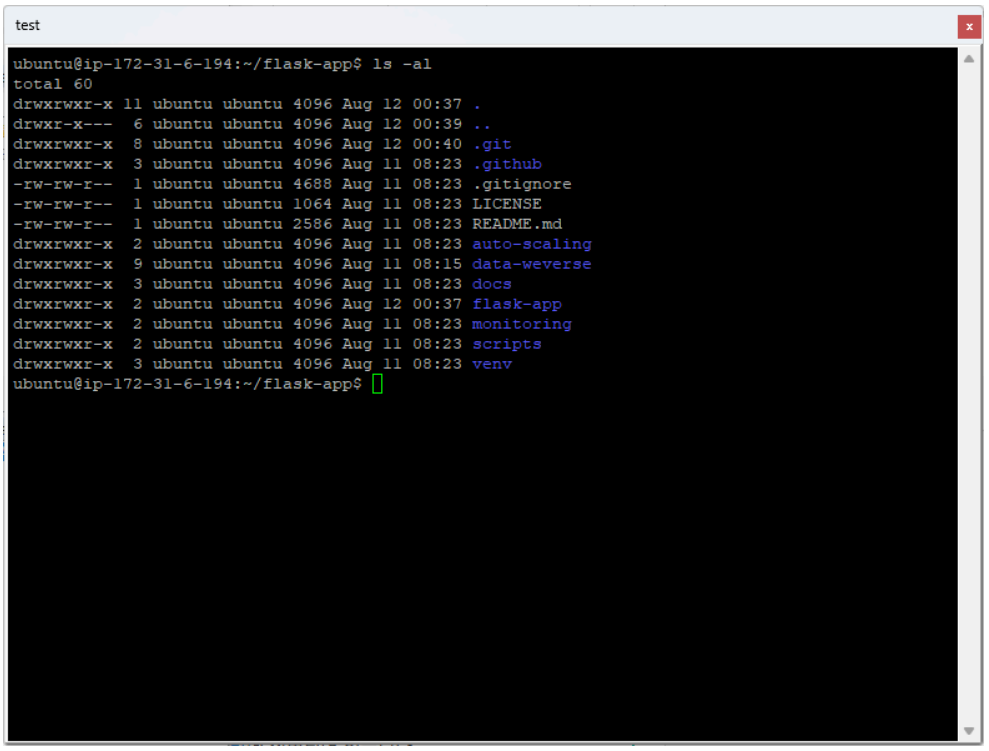
```
sudo apt update
sudo apt install git -y
```

- 깃허브 clone 실행

```
git clone https://github.com/USERNAME/flask-ec2-test.git
#사용 중인 깃허브 주소 입력
```

- 깃허브 이름과 비밀번호 입력
  - 이름: 우측 상단 아이콘 → settings 순서로 클릭하고 name 확인
  - 비밀번호:
    - 우측 상단 아이콘 → settings → 좌측 developer settings 클릭
    - Personal access tokens → Tokens (classic) 클릭
    - Generate new token → Generate new token (classic) 클릭

- Note: 토큰 이름(예: `EC2 Deploy Token`) 입력
- Expiration: 유효기간 설정 (예: 90일, 무기한 필요시 “No expiration” 선택)
- Select scopes: 필요한 권한 체크
  - 기본 push/pull 용: `repo` 체크
- Generate token 버튼 클릭
- 발급된 토큰을 안전한 곳에 복사 (다시 확인 불가)



6. 배포 흐름 간단 정리

1. 로컬 개발
   └ app.py 등 코드 작성/수정

2. GitHub에 업로드 (push)
   └ git add .
   └ git commit -m "메시지"
   └ git push origin <브랜치명>

3. EC2 서버 접속 (SSH)

4. 최신 코드 가져오기 (pull)
   └ cd ~/프로젝트폴더
   └ git pull origin <브랜치명>

5. 애플리케이션 실행
   └ python app.py

6. 브라우저로 확인
   └ http://EC2_PUBLIC_IP:5000

7. CloudWatch 메트릭(기본) 확인 – CPU/네트워크 그래프

1. **AWS Management Console** 로그인
2. **EC2 서비스** 선택
3. 왼쪽 메뉴에서 **인스턴스 → 대상 EC2 선택**
4. **모니터링** 탭 클릭
5. 기본 그래프 확인
   - **CPU Utilization** (CPU 사용률)
   - **Network In / Network Out** (네트워크 입출력)
   - **Disk Read / Disk Write** (디스크 I/O)
6. 필요하면 **시간 범위 조정** (최근 1시간 / 1일 / 1주 등)

